a. Preparation

1) Import Library

In [1]:
```
!pip install gdown
!pip install scikit-learn
!pip install keras
!pip install tensorflow
!pip install seaborn

import os
import zipfile
import gdown
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from PIL import Image
import time
import psutil

from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.decomposition import PCA as RandomizedPCA
from sklearn import svm
from sklearn.svm import SVC
from sklearn.pipeline import make_pipeline
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    precision_score,
    recall_score,
)

import tensorflow as tf
from tensorflow import keras
from keras.models import Sequential, Model
from tensorflow.keras.utils import image_dataset_from_directory
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPla
from tensorflow.keras.metrics import Precision, Recall, F1Score
from keras.layers import (
    Conv2D,
    MaxPooling2D,
    Flatten,
    Dense,
    Dropout,
    BatchNormalization,
    GlobalAveragePooling2D,
    Rescaling
```

```
)
from tensorflow.keras.layers import (
    RandomFlip,
    RandomRotation,
    RandomZoom,
    RandomContrast,
    RandomTranslation
)
from tensorflow.keras import layers, Model
```

```
Requirement already satisfied: gdown in /opt/anaconda3/lib/python3.1
2/site-packages (5.2.0)
Requirement already satisfied: beautifulsoup4 in /opt/anaconda3/lib/
python3.12/site-packages (from gdown) (4.12.3)
Requirement already satisfied: filelock in /opt/anaconda3/lib/python
3.12/site-packages (from gdown) (3.13.1)
Requirement already satisfied: requests[socks] in /opt/anaconda3/li
b/python3.12/site-packages (from gdown) (2.32.3)
Requirement already satisfied: tqdm in /opt/anaconda3/lib/python3.1
2/site-packages (from gdown) (4.66.5)
Requirement already satisfied: soupsieve>1.2 in /opt/anaconda3/lib/p
ython3.12/site-packages (from beautifulsoup4->gdown) (2.5)
Requirement already satisfied: charset-normalizer<4,>=2 in /opt/anac
onda3/lib/python3.12/site-packages (from requests[socks]->gdown) (3.
3.2)
Requirement already satisfied: idna<4,>=2.5 in /opt/anaconda3/lib/py
thon3.12/site-packages (from requests[socks]->gdown) (3.7)
Requirement already satisfied: urllib3<3,>=1.21.1 in /opt/anaconda3/
lib/python3.12/site-packages (from requests[socks]->gdown) (2.2.3)
Requirement already satisfied: certifi>=2017.4.17 in /opt/anaconda3/
lib/python3.12/site-packages (from requests[socks]->gdown) (2025.1.3
1)
Requirement already satisfied: PySocks!=1.5.7,>=1.5.6 in /opt/anacon
da3/lib/python3.12/site-packages (from requests[socks]->gdown) (1.7.
1)
Requirement already satisfied: scikit-learn in /opt/anaconda3/lib/py
thon3.12/site-packages (1.5.1)
Requirement already satisfied: numpy>=1.19.5 in /opt/anaconda3/lib/p
ython3.12/site-packages (from scikit-learn) (1.26.4)
Requirement already satisfied: scipy>=1.6.0 in /opt/anaconda3/lib/py
thon3.12/site-packages (from scikit-learn) (1.13.1)
Requirement already satisfied: joblib>=1.2.0 in /opt/anaconda3/lib/p
ython3.12/site-packages (from scikit-learn) (1.4.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /opt/anaconda
3/lib/python3.12/site-packages (from scikit-learn) (3.5.0)
Requirement already satisfied: keras in /opt/anaconda3/lib/python3.1
2/site-packages (3.9.0)
Requirement already satisfied: absl-py in /opt/anaconda3/lib/python
3.12/site-packages (from keras) (2.2.0)
Requirement already satisfied: numpy in /opt/anaconda3/lib/python3.1
2/site-packages (from keras) (1.26.4)
Requirement already satisfied: rich in /opt/anaconda3/lib/python3.1
2/site-packages (from keras) (13.7.1)
Requirement already satisfied: namex in /opt/anaconda3/lib/python3.1
2/site-packages (from keras) (0.0.8)
Requirement already satisfied: h5py in /opt/anaconda3/lib/python3.1
```

```
2/site-packages (from keras) (3.11.0)
Requirement already satisfied: optree in /opt/anaconda3/lib/python3.
12/site-packages (from keras) (0.14.1)
Requirement already satisfied: ml-dtypes in /opt/anaconda3/lib/pytho
n3.12/site-packages (from keras) (0.3.2)
Requirement already satisfied: packaging in /opt/anaconda3/lib/pytho
n3.12/site-packages (from keras) (24.1)
Requirement already satisfied: typing-extensions>=4.5.0 in /opt/anac
onda3/lib/python3.12/site-packages (from optree->keras) (4.11.0)
Requirement already satisfied: markdown-it-py>=2.2.0 in /opt/anacond
a3/lib/python3.12/site-packages (from rich->keras) (2.2.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /opt/anaco
nda3/lib/python3.12/site-packages (from rich->keras) (2.15.1)
Requirement already satisfied: mdurl~=0.1 in /opt/anaconda3/lib/pyth
on3.12/site-packages (from markdown-it-py>=2.2.0->rich->keras) (0.1.
0)
Requirement already satisfied: tensorflow in /opt/anaconda3/lib/pyth
on3.12/site-packages (2.16.2)
Requirement already satisfied: absl-py>=1.0.0 in /opt/anaconda3/lib/
python3.12/site-packages (from tensorflow) (2.2.0)
Requirement already satisfied: astunparse>=1.6.0 in /opt/anaconda3/l
ib/python3.12/site-packages (from tensorflow) (1.6.3)
Requirement already satisfied: flatbuffers>=23.5.26 in /opt/anaconda
3/lib/python3.12/site-packages (from tensorflow) (25.2.10)
Requirement already satisfied: gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 i
n /opt/anaconda3/lib/python3.12/site-packages (from tensorflow) (0.
6.0)
Requirement already satisfied: google-pasta>=0.1.1 in /opt/anaconda
3/lib/python3.12/site-packages (from tensorflow) (0.2.0)
Requirement already satisfied: h5py>=3.10.0 in /opt/anaconda3/lib/py
thon3.12/site-packages (from tensorflow) (3.11.0)
Requirement already satisfied: libclang>=13.0.0 in /opt/anaconda3/li
b/python3.12/site-packages (from tensorflow) (18.1.1)
Requirement already satisfied: ml-dtypes~=0.3.1 in /opt/anaconda3/li
b/python3.12/site-packages (from tensorflow) (0.3.2)
Requirement already satisfied: opt-einsum>=2.3.2 in /opt/anaconda3/l
ib/python3.12/site-packages (from tensorflow) (3.4.0)
Requirement already satisfied: packaging in /opt/anaconda3/lib/pytho
n3.12/site-packages (from tensorflow) (24.1)
Requirement already satisfied: protobuf!=4.21.0,!=4.21.1,!=4.21.2,!=
4.21.3,!=4.21.4,!=4.21.5,<5.0.0dev,>=3.20.3 in /opt/anaconda3/lib/py
thon3.12/site-packages (from tensorflow) (4.25.3)
Requirement already satisfied: requests<3,>=2.21.0 in /opt/anaconda
3/lib/python3.12/site-packages (from tensorflow) (2.32.3)
Requirement already satisfied: setuptools in /opt/anaconda3/lib/pyth
on3.12/site-packages (from tensorflow) (75.1.0)
Requirement already satisfied: six>=1.12.0 in /opt/anaconda3/lib/pyt
hon3.12/site-packages (from tensorflow) (1.16.0)
Requirement already satisfied: termcolor>=1.1.0 in /opt/anaconda3/li
b/python3.12/site-packages (from tensorflow) (2.5.0)
Requirement already satisfied: typing-extensions>=3.6.6 in /opt/anac
onda3/lib/python3.12/site-packages (from tensorflow) (4.11.0)
Requirement already satisfied: wrapt>=1.11.0 in /opt/anaconda3/lib/p
ython3.12/site-packages (from tensorflow) (1.14.1)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /opt/anaconda
3/lib/python3.12/site-packages (from tensorflow) (1.71.0)
```

```
Requirement already satisfied: tensorboard<2.17,>=2.16 in /opt/anaco
nda3/lib/python3.12/site-packages (from tensorflow) (2.16.2)
Requirement already satisfied: keras>=3.0.0 in /opt/anaconda3/lib/py
thon3.12/site-packages (from tensorflow) (3.9.0)
Requirement already satisfied: numpy<2.0.0,>=1.26.0 in /opt/anaconda
3/lib/python3.12/site-packages (from tensorflow) (1.26.4)
Requirement already satisfied: wheel<1.0,>=0.23.0 in /opt/anaconda3/
lib/python3.12/site-packages (from astunparse>=1.6.0->tensorflow) (
0.44.0)
Requirement already satisfied: rich in /opt/anaconda3/lib/python3.1
2/site-packages (from keras>=3.0.0->tensorflow) (13.7.1)
Requirement already satisfied: namex in /opt/anaconda3/lib/python3.1
2/site-packages (from keras>=3.0.0->tensorflow) (0.0.8)
Requirement already satisfied: optree in /opt/anaconda3/lib/python3.
12/site-packages (from keras>=3.0.0->tensorflow) (0.14.1)
Requirement already satisfied: charset-normalizer<4,>=2 in /opt/anac
onda3/lib/python3.12/site-packages (from requests<3,>=2.21.0->tensor
flow) (3.3.2)
Requirement already satisfied: idna<4,>=2.5 in /opt/anaconda3/lib/py
thon3.12/site-packages (from requests<3,>=2.21.0->tensorflow) (3.7)
Requirement already satisfied: urllib3<3,>=1.21.1 in /opt/anaconda3/
lib/python3.12/site-packages (from requests<3,>=2.21.0->tensorflow)
(2.2.3)
Requirement already satisfied: certifi>=2017.4.17 in /opt/anaconda3/
lib/python3.12/site-packages (from requests<3,>=2.21.0->tensorflow)
(2025.1.31)
Requirement already satisfied: markdown>=2.6.8 in /opt/anaconda3/li
b/python3.12/site-packages (from tensorboard<2.17,>=2.16->tensorflo
w) (3.4.1)
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0
in /opt/anaconda3/lib/python3.12/site-packages (from tensorboard<2.1
7,>=2.16->tensorflow) (0.7.2)
Requirement already satisfied: werkzeug>=1.0.1 in /opt/anaconda3/li
b/python3.12/site-packages (from tensorboard<2.17,>=2.16->tensorflo
w) (3.0.3)
Requirement already satisfied: MarkupSafe>=2.1.1 in /opt/anaconda3/l
ib/python3.12/site-packages (from werkzeug>=1.0.1->tensorboard<2.17,
>=2.16->tensorflow) (2.1.3)
Requirement already satisfied: markdown-it-py>=2.2.0 in /opt/anacond
a3/lib/python3.12/site-packages (from rich->keras>=3.0.0->tensorflo
w) (2.2.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /opt/anaco
nda3/lib/python3.12/site-packages (from rich->keras>=3.0.0->tensorfl
ow) (2.15.1)
Requirement already satisfied: mdurl~=0.1 in /opt/anaconda3/lib/pyth
on3.12/site-packages (from markdown-it-py>=2.2.0->rich->keras>=3.0.0
->tensorflow) (0.1.0)
Requirement already satisfied: seaborn in /opt/anaconda3/lib/python
3.12/site-packages (0.13.2)
Requirement already satisfied: numpy!=1.24.0,>=1.20 in /opt/anaconda
3/lib/python3.12/site-packages (from seaborn) (1.26.4)
Requirement already satisfied: pandas>=1.2 in /opt/anaconda3/lib/pyt
hon3.12/site-packages (from seaborn) (2.2.2)
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in /opt/anaco
nda3/lib/python3.12/site-packages (from seaborn) (3.9.2)
Requirement already satisfied: contourpy>=1.0.1 in /opt/anaconda3/li
```

```
b/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (
1.2.0)
Requirement already satisfied: cycler>=0.10 in /opt/anaconda3/lib/py
thon3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.1
1.0)
Requirement already satisfied: fonttools>=4.22.0 in /opt/anaconda3/l
ib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn)
(4.51.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /opt/anaconda3/l
ib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn)
(1.4.4)
Requirement already satisfied: packaging>=20.0 in /opt/anaconda3/li
b/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (
24.1)
Requirement already satisfied: pillow>=8 in /opt/anaconda3/lib/pytho
n3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (10.4.0)
Requirement already satisfied: pyparsing>=2.3.1 in /opt/anaconda3/li
b/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (
3.1.2)
Requirement already satisfied: python-dateutil>=2.7 in /opt/anaconda
3/lib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seabor
n) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /opt/anaconda3/lib/py
thon3.12/site-packages (from pandas>=1.2->seaborn) (2024.1)
Requirement already satisfied: tzdata>=2022.7 in /opt/anaconda3/lib/
python3.12/site-packages (from pandas>=1.2->seaborn) (2023.3)
Requirement already satisfied: six>=1.5 in /opt/anaconda3/lib/python
3.12/site-packages (from python-dateutil>=2.7->matplotlib!=3.6.1,>=
3.4->seaborn) (1.16.0)
```

2) Load Dataset

In [2]:
```python
# 1) Download image zip file from Google Drive
url = "https://drive.google.com/uc?id=1Xsfvg57L2j44AHAIomttWTlLlyzS
output = "waste_images.zip"
gdown.download(url, output, quiet=False)

# 2) Extract it into local folder
with zipfile.ZipFile(output, "r") as zip_ref:
    zip_ref.extractall("waste_images")

# 3) Set the base directory
base_dir = "waste_images"
images_dir = os.path.join(base_dir, "images")
```

```
Downloading...
From (original): https://drive.google.com/uc?id=1Xsfvg57L2j44AHAIomt
tWTlLlyzS_8Gf
From (redirected): https://drive.google.com/uc?id=1Xsfvg57L2j44AHAIo
mttWTlLlyzS_8Gf&confirm=t&uuid=ba3c4fc2-1a40-4016-a1d5-28c92e4cbca1
To: /Users/meng/Desktop/CBS courses/semester_2/Machine & DeepLearnin
g/ml_final_project/waste_images.zip
100%|██████████| 971M/971M [00:46<00:00, 21.1MB/s]
```

3) Read Dataset

In [3]:
```
# 1) Define a function to get image paths and labels
def read_files():
    images=[]
    labels=[]
    main_path=images_dir

    for category in os.listdir(main_path):
        if category != '.DS_Store': # .DS_Store is a default file o

            # Every folder has 2 sub-folders: 'real_world' & 'defau
            for file in os.listdir(os.path.join(main_path,category,
                images.append(os.path.join(main_path,category,'real
                labels.append(category)

            for file in os.listdir(os.path.join(main_path,category,
                images.append(os.path.join(main_path,category,'defa
                labels.append(category)

    return images,labels
```

In [4]:
```
# 2) Read the data
images, labels = read_files()

# 3) Preview some data
images[:5], labels[:5]
```

Out[4]:
```
(['waste_images/images/disposable_plastic_cutlery/real_world/Image
_15.png',
  'waste_images/images/disposable_plastic_cutlery/real_world/Image
_29.png',
  'waste_images/images/disposable_plastic_cutlery/real_world/Image
_178.png',
  'waste_images/images/disposable_plastic_cutlery/real_world/Image
_144.png',
  'waste_images/images/disposable_plastic_cutlery/real_world/Image
_150.png'],
 ['disposable_plastic_cutlery',
  'disposable_plastic_cutlery',
  'disposable_plastic_cutlery',
  'disposable_plastic_cutlery',
  'disposable_plastic_cutlery'])
```

4) GPU Tracking (if GPUs enable)

In [5]:
```
def get_gpu_memory():
    gpus = tf.config.list_physical_devices('GPU')
    if gpus:
        try:
            info = tf.config.experimental.get_memory_info(gpus[0].n
            return info['current']  # in bytes
        except:
            return None
    return None
```

b. EDA

1) Basic Information

In [6]:
```
# check the number of images
print(f"Number of images: {len(images)}")

# check the number of labels
print(f"Number of labels: {len(labels)}")

# check the number of unique labels
print(f"Number of unique labels: {len(set(labels))}")

# check the name of the labels
print(f"Labels: {set(labels)}")
```

```
Number of images: 15000
Number of labels: 15000
Number of unique labels: 30
Labels: {'plastic_straws', 'office_paper', 'steel_food_cans', 'plast
ic_soda_bottles', 'styrofoam_cups', 'plastic_trash_bags', 'coffee_gr
ounds', 'disposable_plastic_cutlery', 'plastic_cup_lids', 'glass_foo
d_jars', 'plastic_detergent_bottles', 'plastic_shopping_bags', 'news
paper', 'tea_bags', 'glass_cosmetic_containers', 'clothing', 'cardbo
ard_boxes', 'styrofoam_food_containers', 'food_waste', 'eggshells',
'aluminum_soda_cans', 'shoes', 'paper_cups', 'plastic_water_bottle
s', 'magazines', 'plastic_food_containers', 'aerosol_cans', 'glass_b
everage_bottles', 'cardboard_packaging', 'aluminum_food_cans'}
```

In [7]:
```
# Check the size of each image
# 1) Create a function to return the image size
def check_image_sizes(folder_path):
    image_sizes = {}

    for root, dirs, files in os.walk(folder_path):
        for file in files:
            if file.lower().endswith((".jpg", ".jpeg", ".png", ".gi
                image_path = os.path.join(root, file)
                with Image.open(image_path) as img:
                    width, height = img.size
                    if (width, height) in image_sizes:
                        image_sizes[(width, height)].append(image_p
                    else:
                        image_sizes[(width, height)] = [image_path]

    return image_sizes

# 2) Apply the imgae size checking function
image_sizes = check_image_sizes(images_dir)

# 3) Print the image size
print("IMAGE SIZES:")
for size, images in image_sizes.items():
    print(f"Size: {size}, Number of Images: {len(images)}")
```

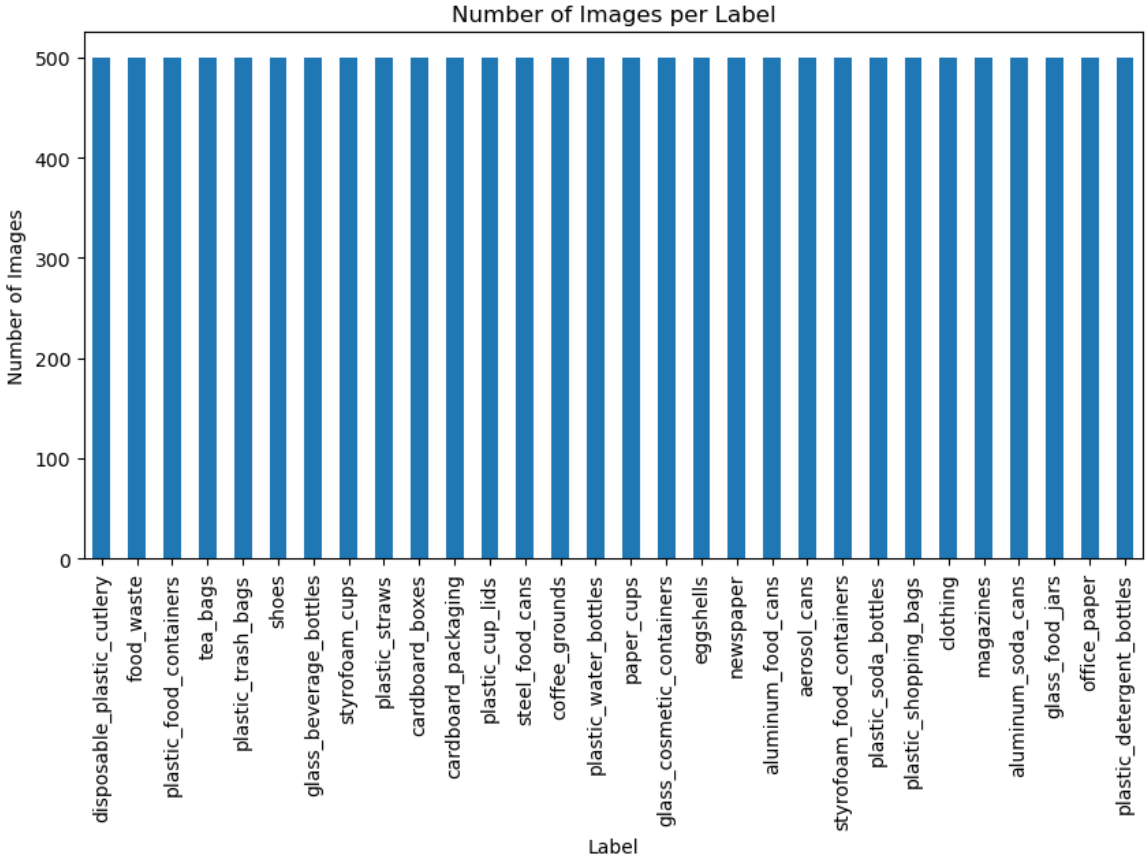
```
IMAGE SIZES:
Size: (256, 256), Number of Images: 15000
```

2) Data Distribution

In [8]:
```python
# Plot the amount of images per label to check if the data is balan

def plot_images_per_label(labels):
    label_counts = pd.Series(labels).value_counts()
    plt.figure(figsize=(10, 5))
    label_counts.plot(kind='bar')
    plt.title('Number of Images per Label')
    plt.xlabel('Label')
    plt.ylabel('Number of Images')
    plt.show()

plot_images_per_label(labels)
```



3) Random Image Check

In [9]:
```python
# Plot random images
def plot_random_images(images, labels):
    random_indices = np.random.choice(len(images), 9, replace=False
    plt.figure(figsize=(10, 10))
    for i in range(9):
        plt.subplot(3, 3, i + 1)
        img = tf.keras.preprocessing.image.load_img(images[random_i
        plt.imshow(img)
        plt.title(labels[random_indices[i]])
```

```
        plt.axis('off')
    plt.show()

plot_random_images(images, labels)
```



c. Models

1. SVM (Baseline Model)

SVM (Baseline Model) - Prepare the Dataset

In [10]:
```
# Load the images and labels
X = []
y = []

for i in range(len(images)):
    img = keras.utils.load_img(images[i], target_size=(32,32))
    img = keras.utils.img_to_array(img)
```

```
    X.append(img)
    y.append(labels[i])
```

In [11]:
```
# Convert the images to numpy array
X = np.array(X)
# Convert the labels to numpy array
y = np.array(y)

# Check the shape of the images
print(f"Shape of images: {X.shape}")
# Check the shape of the labels
print(f"Shape of labels: {y.shape}")
```

```
Shape of images: (15000, 32, 32, 3)
Shape of labels: (15000,)
```

In [12]:
```
# Encode the labels
label_encoder = LabelEncoder()
y_encoded = label_encoder.fit_transform(y)

# check the shape of the encoded labels
print(f"Shape of encoded labels: {y_encoded.shape}")
# check the number of unique labels
print(f"Number of unique labels: {len(set(y_encoded))}")
```

```
Shape of encoded labels: (15000,)
Number of unique labels: 30
```

In [13]:
```
# Split the data into train, test and validation sets
# 70% training, 15% testing, 15% validation
X_train, X_temp, y_train, y_temp = train_test_split(X, y_encoded, t
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, tes

# Check the shape of the train, test and validation sets
print(f"Shape of training set: {X_train.shape}")
print(f"Shape of testing set: {X_test.shape}")
print(f"Shape of validation set: {X_val.shape}")
```

```
Shape of training set: (10500, 32, 32, 3)
Shape of testing set: (2250, 32, 32, 3)
Shape of validation set: (2250, 32, 32, 3)
```

SVM (Baseline Model) - Pre-processing

In [14]:
```
# Pre-process the data
# 1) Normalize image pixels
X_train_svm = X_train.astype('float32') / 255.0
X_val_svm = X_val.astype('float32') / 255.0
X_test_svm = X_test.astype('float32') / 255.0

# 2-1) Flatten images
X_train_svm = X_train_svm.reshape(X_train_svm.shape[0], -1)
X_val_svm = X_val_svm.reshape(X_val_svm.shape[0], -1)
X_test_svm = X_test_svm.reshape(X_test_svm.shape[0], -1)

# 2-2) Check the shape of the images after flattening
```

```
print(f"Shape of training set after flattening: {X_train_svm.shape}
print(f"Shape of testing set after flattening: {X_test_svm.shape}")
print(f"Shape of validation set after flattening: {X_val_svm.shape}
```

```
Shape of training set after flattening: (10500, 3072)
Shape of testing set after flattening: (2250, 3072)
Shape of validation set after flattening: (2250, 3072)
```

In [15]:
```
# 3) Scale images
scaler = StandardScaler()
X_train_svm = scaler.fit_transform(X_train_svm)
X_val_svm = scaler.transform(X_val_svm)
X_test_svm = scaler.transform(X_test_svm)
```

SVM (Baseline Model) - Building the Model

In [16]:
```
# Use principal component analysis to extract 150 fundamental compo
pca = RandomizedPCA(n_components=300, whiten=True, random_state=42)

# Instantiate SVC model with 'rbf' kernel
svc = svm.SVC(kernel="rbf", class_weight="balanced")

# Combine them into a pipeline
model = make_pipeline(pca, svc)
```

In [17]:
```
# Tune SVM hyperparameters using GridSearchCV
# 1) Set testing parameters for GridSearchCV
param_grid = {"svc__C": [1, 5, 10, 50], "svc__gamma": ['scale', 0.0

# 2) Apply GridSearchCV function
grid = GridSearchCV(model, param_grid, cv=5, n_jobs=-1, verbose=2)
```

SVM (Baseline Model) - Training the Model

In [18]:
```
# Train the model
# 1) Record memory usage before training
mem_before_svm = psutil.virtual_memory().used
#gpu_mem_before_svm = get_gpu_memory()

# 2) Record training start time
start_time_svm = time.time()

# 3) Fit the data into SVM model in GridSearchCV
grid.fit(X_train_svm, y_train)
print("Best parameters:", grid.best_params_)

# 4) Record training end time
end_time_svm = time.time()

# 5) Record memory usage after training
mem_after_svm = psutil.virtual_memory().used
#gpu_mem_after_svm = get_gpu_memory()
```

```
Fitting 5 folds for each of 12 candidates, totalling 60 fits
```

```
[CV] END .....................svc__C=1, svc__gamma=scale; total
time=  27.3s
[CV] END .....................svc__C=1, svc__gamma=scale; total
time=  31.2s
[CV] END .....................svc__C=1, svc__gamma=scale; total
time=  31.4s
[CV] END .....................svc__C=1, svc__gamma=scale; total
time=  31.7s
[CV] END .....................svc__C=1, svc__gamma=scale; total
time=  31.9s
[CV] END .....................svc__C=1, svc__gamma=0.01; total
time=  32.2s
[CV] END .....................svc__C=1, svc__gamma=0.01; total
time=  32.2s
[CV] END .....................svc__C=1, svc__gamma=0.01; total
time=  32.4s
[CV] END .....................svc__C=1, svc__gamma=0.01; total
time=  32.7s
[CV] END .....................svc__C=1, svc__gamma=0.01; total
time=  33.4s
[CV] END .....................svc__C=1, svc__gamma=0.001; total
time=  32.8s
[CV] END .....................svc__C=1, svc__gamma=0.001; total
time=  32.1s
[CV] END .....................svc__C=1, svc__gamma=0.001; total
time=  32.7s
[CV] END .....................svc__C=1, svc__gamma=0.001; total
time=  32.8s
[CV] END .....................svc__C=5, svc__gamma=scale; total
time=  31.8s
[CV] END .....................svc__C=1, svc__gamma=0.001; total
time=  32.9s
[CV] END .....................svc__C=5, svc__gamma=scale; total
time=  32.3s
[CV] END .....................svc__C=5, svc__gamma=scale; total
time=  32.2s
[CV] END .....................svc__C=5, svc__gamma=scale; total
time=  32.7s
[CV] END .....................svc__C=5, svc__gamma=scale; total
time=  32.3s
[CV] END .....................svc__C=5, svc__gamma=0.01; total
time=  34.0s
[CV] END .....................svc__C=5, svc__gamma=0.001; total
time=  31.2s
[CV] END .....................svc__C=5, svc__gamma=0.001; total
time=  31.5s
[CV] END .....................svc__C=5, svc__gamma=0.001; total
time=  32.2s
[CV] END .....................svc__C=5, svc__gamma=0.001; total
time=  31.1s
[CV] END .....................svc__C=5, svc__gamma=0.001; total
time=  32.5s
[CV] END .....................svc__C=5, svc__gamma=0.01; total
time=  34.2s
[CV] END .....................svc__C=5, svc__gamma=0.01; total
time=  34.3s
```

```
[CV] END ..........................svc__C=5, svc__gamma=0.01; total
time=  35.2s
[CV] END ..........................svc__C=5, svc__gamma=0.01; total
time=  36.1s
[CV] END ........................svc__C=10, svc__gamma=scale; total
time=  34.7s
[CV] END ........................svc__C=10, svc__gamma=scale; total
time=  34.7s
[CV] END ........................svc__C=10, svc__gamma=scale; total
time=  35.5s
[CV] END ........................svc__C=10, svc__gamma=scale; total
time=  35.9s
[CV] END ........................svc__C=10, svc__gamma=scale; total
time=  35.9s
[CV] END ........................svc__C=10, svc__gamma=0.01; total
time=  35.2s
[CV] END ........................svc__C=10, svc__gamma=0.01; total
time=  36.1s
[CV] END ........................svc__C=10, svc__gamma=0.01; total
time=  36.9s
[CV] END ........................svc__C=10, svc__gamma=0.01; total
time=  36.1s
[CV] END ........................svc__C=10, svc__gamma=0.01; total
time=  36.5s
[CV] END ........................svc__C=10, svc__gamma=0.001; total
time=  36.4s
[CV] END ........................svc__C=10, svc__gamma=0.001; total
time=  36.0s
[CV] END ........................svc__C=10, svc__gamma=0.001; total
time=  36.4s
[CV] END ........................svc__C=10, svc__gamma=0.001; total
time=  35.2s
[CV] END ........................svc__C=10, svc__gamma=0.001; total
time=  36.7s
[CV] END ........................svc__C=50, svc__gamma=scale; total
time=  37.4s
[CV] END ........................svc__C=50, svc__gamma=scale; total
time=  37.2s
[CV] END ........................svc__C=50, svc__gamma=scale; total
time=  36.7s
[CV] END ........................svc__C=50, svc__gamma=scale; total
time=  38.1s
[CV] END ........................svc__C=50, svc__gamma=scale; total
time=  37.9s
[CV] END ........................svc__C=50, svc__gamma=0.01; total
time=  37.4s
[CV] END ........................svc__C=50, svc__gamma=0.01; total
time=  36.9s
[CV] END ........................svc__C=50, svc__gamma=0.01; total
time=  36.7s
[CV] END ........................svc__C=50, svc__gamma=0.01; total
time=  37.5s
[CV] END ........................svc__C=50, svc__gamma=0.001; total
time=  35.7s
[CV] END ........................svc__C=50, svc__gamma=0.001; total
time=  35.5s
```

```
[CV] END .......................svc__C=50, svc__gamma=0.01; total
time=  37.8s
[CV] END .......................svc__C=50, svc__gamma=0.001; total
time=  36.1s
[CV] END .......................svc__C=50, svc__gamma=0.001; total
time=  35.5s
[CV] END .......................svc__C=50, svc__gamma=0.001; total
time=  34.5s
Best parameters: {'svc__C': 50, 'svc__gamma': 'scale'}
```

In [19]:
```python
# 6) Training time used
train_duration_svm = end_time_svm - start_time_svm
print(f"Training Time: {train_duration_svm:.2f} sec")

# 7) CPU & GPU usage
mem_used_svm = (mem_after_svm - mem_before_svm) / 1024**2  # MB
print(f"CPU Memory Used for Training: {mem_used_svm:.2f} MB")

# if gpu_mem_before_svm is not None and gpu_mem_after_svm is not No
#   gpu_mem_used_svm = (gpu_mem_after_svm - gpu_mem_before_svm) / 1

#   print(f"GPU Memory Used for Training: {gpu_mem_used_svm:.2f} MB

# else:
#   print("GPU memory info not available.")
```

```
Training Time: 253.39 sec
CPU Memory Used for Training: 105.09 MB
```

SVM (Baseline Model) - Making Predictions

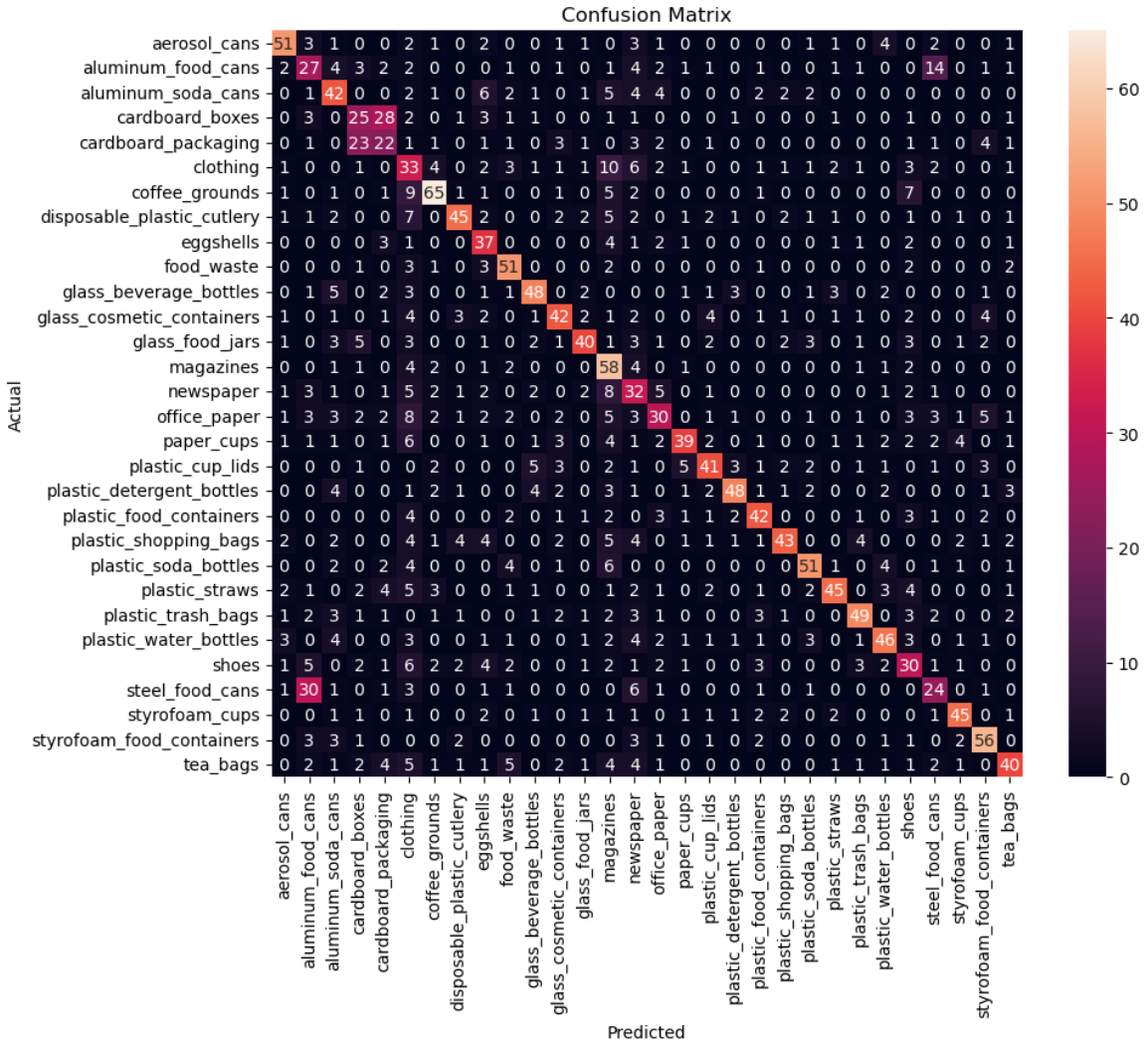
In [20]:
```python
# Predictions
model = grid.best_estimator_
y_pred = model.predict(X_test_svm)
```

SVM (Baseline Model) - Evaluating the model

In [21]:
```python
# 1) Classification Report
print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	0.72	0.68	0.70	75
1	0.31	0.39	0.34	70
2	0.49	0.56	0.52	75
3	0.35	0.36	0.35	70
4	0.29	0.33	0.31	67
5	0.25	0.42	0.32	78
6	0.71	0.68	0.70	95
7	0.71	0.56	0.63	80
8	0.46	0.69	0.55	54
9	0.64	0.77	0.70	66
10	0.70	0.64	0.67	75
11	0.60	0.57	0.58	74
12	0.69	0.53	0.60	75
13	0.41	0.74	0.53	78
14	0.32	0.46	0.37	70
15	0.48	0.36	0.41	83
16	0.70	0.51	0.59	77
17	0.63	0.55	0.59	74
18	0.77	0.61	0.68	79
19	0.64	0.64	0.64	66
20	0.75	0.51	0.61	84
21	0.72	0.65	0.68	78
22	0.74	0.56	0.63	81
23	0.72	0.61	0.66	80
24	0.66	0.57	0.61	80
25	0.39	0.42	0.41	72
26	0.41	0.33	0.37	72
27	0.75	0.69	0.72	65
28	0.68	0.74	0.71	76
29	0.66	0.49	0.56	81
accuracy			0.55	2250
macro avg	0.58	0.55	0.56	2250
weighted avg	0.58	0.55	0.56	2250

In [22]:
```
# 2) Confusion Matrix
conf_matrix = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(10, 8))
sns.heatmap(conf_matrix, annot=True, fmt="d", xticklabels=label_enc
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```



2. CNN

CNN - Prepare the Dataset

In [23]:
```
# 1) Get main image dir and categories
path = os.path.join("waste_images", "images")
categories = os.listdir(path)

print(f"Path: {path}")
print(f"Categories: {categories}")
```

```
Path: waste_images/images
Categories: ['disposable_plastic_cutlery', 'food_waste', '.DS_Stor
e', 'office_paper', 'glass_food_jars', 'aluminum_soda_cans', 'magazi
nes', 'clothing', 'plastic_shopping_bags', 'plastic_soda_bottles', '
styrofoam_food_containers', 'aerosol_cans', 'aluminum_food_cans', 'n
ewspaper', 'eggshells', 'glass_cosmetic_containers', 'paper_cups', '
plastic_water_bottles', 'coffee_grounds', 'steel_food_cans', 'plasti
c_cup_lids', 'cardboard_packaging', 'cardboard_boxes', 'plastic_stra
ws', 'styrofoam_cups', 'glass_beverage_bottles', 'shoes', 'plastic_t
rash_bags', 'tea_bags', 'plastic_food_containers', 'plastic_detergen
t_bottles']
```

In [24]:
```
# 2) Define the batch size and image size
```

```
BATCH_SIZE = 32
IMG_SIZE = (256, 256)

# 3) Define the training dataset
train = image_dataset_from_directory(
    path,
    labels='inferred',  # Automatically infers labels from subdirec
    label_mode='categorical',  # Return labels as one-hot encoded v
    shuffle=True,
    batch_size=BATCH_SIZE,
    image_size=IMG_SIZE,
    validation_split=0.3,  # 30% for validation
    subset='training',
    seed=42
)

class_names = train.class_names

# 4) Define the temporary validation dataset
temp_validation = image_dataset_from_directory(
    path,
    labels='inferred',  # Automatically infers labels from subdirec
    label_mode='categorical',  # Return labels as one-hot encoded v
    shuffle=True,
    batch_size=BATCH_SIZE,
    image_size=IMG_SIZE,
    validation_split=0.3,  # 30% for validation
    subset='validation',
    seed=42
)

# 5) Split the temporary validation dataset into validation and tes
# Define the ratio for splitting the temporary validation dataset
val_ratio = 0.5
val_size = int(len(temp_validation) * val_ratio)

# Split the the temporary validation dataset
validation = temp_validation.take(val_size)
test = temp_validation.skip(val_size)

# Auto-tune the dataset for training performance by using pre-fetch
AUTOTUNE = tf.data.experimental.AUTOTUNE
train = train.prefetch(buffer_size=AUTOTUNE)
```

```
Found 15000 files belonging to 30 classes.
Using 10500 files for training.
```

```
2025-05-12 12:52:59.632462: I metal_plugin/src/device/metal_device.c
c:1154] Metal device set to: Apple M1 Pro
2025-05-12 12:52:59.632872: I metal_plugin/src/device/metal_device.c
c:296] systemMemory: 16.00 GB
2025-05-12 12:52:59.632888: I metal_plugin/src/device/metal_device.c
c:313] maxCacheSize: 5.33 GB
2025-05-12 12:52:59.633257: I tensorflow/core/common_runtime/pluggab
le_device/pluggable_device_factory.cc:305] Could not identify NUMA n
ode of platform GPU ID 0, defaulting to 0. Your kernel may not have
been built with NUMA support.
2025-05-12 12:52:59.633706: I tensorflow/core/common_runtime/pluggab
le_device/pluggable_device_factory.cc:271] Created TensorFlow device
(/job:localhost/replica:0/task:0/device:GPU:0 with 0 MB memory) -> p
hysical PluggableDevice (device: 0, name: METAL, pci bus id: <undefi
ned>)
Found 15000 files belonging to 30 classes.
Using 4500 files for validation.
```

CNN - Pre-processing

In [25]:
```python
# 1) Data augmentation and normalize image pixels for training data
# Define the data augmentation layers
data_augmentation = tf.keras.Sequential([
    keras.layers.RandomFlip("horizontal"),
    # Disabled the following items because they'll lower accuracy
    # keras.layers.RandomRotation(0.05),
    # keras.layers.RandomZoom(0.05),
    # keras.layers.RandomContrast(0.05),
    # keras.layers.RandomBrightness(0.02)
], name="data_augmentation")

# Define pre-processing function with augmentation
def process_image_augmented(image, label):
    image = tf.cast(image, tf.float32) / 255.0
    image = data_augmentation(image)

    return image, label

# Apply pre-processing function on training dataset
train_cnn = train.map(process_image_augmented, num_parallel_calls=A
```

In [26]:
```python
# 2) Normalize image pixels for validation and test datasets
validation_cnn = validation.map(lambda x, y: (x / 255.0, y))
test_cnn = test.map(lambda x, y: (x / 255.0, y))
```

CNN - Building the Model

In [27]:
```python
# Input layer
input = keras.layers.Input(shape=(256, 256, 3), name='input_layer')

# 1st Convolutional Block (32 filters)
# - Conv2D scans images for patterns -> to extract features
# - MaxPooling2D reduces imgae sizes -> to keep the strongest featu
```

```
x = keras.layers.Conv2D(filters=32, kernel_size=(3, 3), activation=
# filter x 32; kernal (filter) size (3, 3)
# activation='relu' -> Rectifies liner Unit, output range [0, ∞); a
# padding='same' -> keep paddings = keep original image size, other
# if not keeping the original image size -> output_size = input_siz

x = keras.layers.MaxPooling2D(pool_size=(2, 2), name='pool1')(x)
# reduced size (2, 2) -> halfen the size

# 2nd Convolutional Block (64 filters)
x = keras.layers.Conv2D(filters=64, kernel_size=(3, 3), activation=
x = keras.layers.MaxPooling2D(pool_size=(2, 2), name='pool2')(x)

# Flatten Layer
x = keras.layers.Flatten(name='flatten')(x)

# Dense Layer
x = keras.layers.Dense(units=128, activation='relu', name='dense1')

# Output Logits layer (30 units for 30 waste categories)
output_logits = keras.layers.Dense(units=30, name='output_logits')(

# Separate Softmax Layer - Turn values into probabilities; sum = 1
output_softmax = keras.layers.Softmax(name='output_softmax')(output

# Build the Model
base_cnn_model = keras.Model(inputs=input, outputs=output_softmax,

# Print Model summary
base_cnn_model.summary()
```

Model: "baseline_cnn"

Layer (type)	Output Shape	
input_layer (InputLayer)	(None, 256, 256, 3)	
conv1 (Conv2D)	(None, 256, 256, 32)	
pool1 (MaxPooling2D)	(None, 128, 128, 32)	
conv2 (Conv2D)	(None, 128, 128, 64)	
pool2 (MaxPooling2D)	(None, 64, 64, 64)	
flatten (Flatten)	(None, 262144)	
dense1 (Dense)	(None, 128)	
output_logits (Dense)	(None, 30)	
output_softmax (Softmax)	(None, 30)	

Total params: 33,577,822 (128.09 MB)

Trainable params: 33,577,822 (128.09 MB)

Non-trainable params: 0 (0.00 B)

CNN - Compiling the Model

In [28]:
```
# Compile the model
base_cnn_model.compile(
    optimizer = 'adam',
    loss = 'categorical_crossentropy',
    metrics = [
        'accuracy',
        Precision(name='precision'),
        Recall(name='recall'),
        F1Score(name='f1_score', average='macro'
        ),
    ]
)
```

CNN - Training the Model

In [29]:
```
# Train the model
# 1) Record memory usage before training
mem_before_base_cnn = psutil.virtual_memory().used
# gpu_mem_before_base_cnn = get_gpu_memory()

# 2) Record training start time
start_time_base_cnn = time.time()

# 3) Fit the data into base CNN model
history = base_cnn_model.fit(
    train_cnn,                          # Training Dataset
    validation_data = validation_cnn,  # Validation Dataset
    epochs = 10,                        # Number of epochs for train
    verbose = 1                         # Showing training progress
)

# 4) Record training end time
end_time_base_cnn = time.time()

# 5) Record memory usage after training
mem_after_base_cnn = psutil.virtual_memory().used
# gpu_mem_after_base_cnn = get_gpu_memory()
```

```
Epoch 1/10
2025-05-12 12:53:00.818616: I tensorflow/core/grappler/optimizers/cu
stom_graph_optimizer_registry.cc:117] Plugin optimizer for device_ty
pe GPU is enabled.
2025-05-12 12:53:00.824685: E tensorflow/core/grappler/optimizers/me
ta_optimizer.cc:961] PluggableGraphOptimizer failed: INVALID_ARGUMEN
T: Failed to deserialize the `graph_buf`.
```

```
329/329 ━━━━━━━━━━━━━━━━━━━━ 40s 116ms/step - accuracy: 0.1391 - f1_
score: 0.1203 - loss: 4.1611 - precision: 0.3023 - recall: 0.0180 -
val_accuracy: 0.3263 - val_f1_score: 0.3169 - val_loss: 2.4801 - val
_precision: 0.6636 - val_recall: 0.1295
Epoch 2/10
329/329 ━━━━━━━━━━━━━━━━━━━━ 0s 105ms/step - accuracy: 0.4412 - f1_s
core: 0.4361 - loss: 2.0589 - precision: 0.7854 - recall: 0.2225
2025-05-12 12:54:15.479447: E tensorflow/core/grappler/optimizers/me
ta_optimizer.cc:961] PluggableGraphOptimizer failed: INVALID_ARGUMEN
T: Failed to deserialize the `graph_buf`.
329/329 ━━━━━━━━━━━━━━━━━━━━ 37s 111ms/step - accuracy: 0.4413 - f1_
score: 0.4363 - loss: 2.0582 - precision: 0.7855 - recall: 0.2228 -
val_accuracy: 0.4585 - val_f1_score: 0.4616 - val_loss: 2.0504 - val
_precision: 0.7480 - val_recall: 0.2888
Epoch 3/10
329/329 ━━━━━━━━━━━━━━━━━━━━ 36s 110ms/step - accuracy: 0.6500 - f1_
score: 0.6483 - loss: 1.2761 - precision: 0.8617 - recall: 0.4887 -
val_accuracy: 0.5272 - val_f1_score: 0.5233 - val_loss: 1.9499 - val
_precision: 0.7329 - val_recall: 0.4201
Epoch 4/10
329/329 ━━━━━━━━━━━━━━━━━━━━ 0s 105ms/step - accuracy: 0.7802 - f1_s
core: 0.7768 - loss: 0.8418 - precision: 0.9039 - recall: 0.6773
2025-05-12 12:55:28.613607: E tensorflow/core/grappler/optimizers/me
ta_optimizer.cc:961] PluggableGraphOptimizer failed: INVALID_ARGUMEN
T: Failed to deserialize the `graph_buf`.
329/329 ━━━━━━━━━━━━━━━━━━━━ 37s 112ms/step - accuracy: 0.7803 - f1_
score: 0.7769 - loss: 0.8415 - precision: 0.9039 - recall: 0.6774 -
val_accuracy: 0.5576 - val_f1_score: 0.5549 - val_loss: 1.9944 - val
_precision: 0.7125 - val_recall: 0.4790
Epoch 5/10
329/329 ━━━━━━━━━━━━━━━━━━━━ 36s 110ms/step - accuracy: 0.8552 - f1_
score: 0.8539 - loss: 0.5839 - precision: 0.9263 - recall: 0.7912 -
val_accuracy: 0.5737 - val_f1_score: 0.5727 - val_loss: 2.0353 - val
_precision: 0.6994 - val_recall: 0.5049
Epoch 6/10
329/329 ━━━━━━━━━━━━━━━━━━━━ 0s 106ms/step - accuracy: 0.8931 - f1_s
core: 0.8912 - loss: 0.3998 - precision: 0.9330 - recall: 0.8532
2025-05-12 12:56:42.300041: E tensorflow/core/grappler/optimizers/me
ta_optimizer.cc:961] PluggableGraphOptimizer failed: INVALID_ARGUMEN
T: Failed to deserialize the `graph_buf`.
329/329 ━━━━━━━━━━━━━━━━━━━━ 37s 113ms/step - accuracy: 0.8931 - f1_
score: 0.8912 - loss: 0.3997 - precision: 0.9331 - recall: 0.8532 -
val_accuracy: 0.6018 - val_f1_score: 0.5970 - val_loss: 1.9556 - val
_precision: 0.7156 - val_recall: 0.5527
Epoch 7/10
329/329 ━━━━━━━━━━━━━━━━━━━━ 36s 111ms/step - accuracy: 0.9282 - f1_
score: 0.9261 - loss: 0.2859 - precision: 0.9493 - recall: 0.9039 -
val_accuracy: 0.6094 - val_f1_score: 0.6076 - val_loss: 2.1730 - val
_precision: 0.6970 - val_recall: 0.5719
Epoch 8/10
329/329 ━━━━━━━━━━━━━━━━━━━━ 0s 106ms/step - accuracy: 0.9335 - f1_s
core: 0.9310 - loss: 0.2618 - precision: 0.9518 - recall: 0.9149
2025-05-12 12:57:55.796652: E tensorflow/core/grappler/optimizers/me
ta_optimizer.cc:961] PluggableGraphOptimizer failed: INVALID_ARGUMEN
T: Failed to deserialize the `graph_buf`.
```

```
329/329 ━━━━━━━━━━━━━━━━━━━━ 37s 112ms/step - accuracy: 0.9335 - f1_
score: 0.9310 - loss: 0.2617 - precision: 0.9518 - recall: 0.9149 -
val_accuracy: 0.6000 - val_f1_score: 0.5962 - val_loss: 2.2994 - val
_precision: 0.6768 - val_recall: 0.5683
Epoch 9/10
329/329 ━━━━━━━━━━━━━━━━━━━━ 37s 111ms/step - accuracy: 0.9492 - f1_
score: 0.9461 - loss: 0.1883 - precision: 0.9615 - recall: 0.9374 -
val_accuracy: 0.5911 - val_f1_score: 0.5907 - val_loss: 2.3955 - val
_precision: 0.6661 - val_recall: 0.5656
Epoch 10/10
329/329 ━━━━━━━━━━━━━━━━━━━━ 0s 106ms/step - accuracy: 0.9538 - f1_s
core: 0.9515 - loss: 0.1818 - precision: 0.9608 - recall: 0.9461
2025-05-12 12:59:09.463673: E tensorflow/core/grappler/optimizers/me
ta_optimizer.cc:961] PluggableGraphOptimizer failed: INVALID_ARGUMEN
T: Failed to deserialize the `graph_buf`.
329/329 ━━━━━━━━━━━━━━━━━━━━ 37s 112ms/step - accuracy: 0.9538 - f1_
score: 0.9515 - loss: 0.1818 - precision: 0.9608 - recall: 0.9461 -
val_accuracy: 0.5933 - val_f1_score: 0.5893 - val_loss: 2.4280 - val
_precision: 0.6693 - val_recall: 0.5647
```

In [30]:

```
# 6) Training time used
train_duration_base_cnn = end_time_base_cnn - start_time_base_cnn
print(f"Training Time: {train_duration_base_cnn:.2f} sec")

# 7) CPU & GPU usage
mem_used_base_cnn = (mem_after_base_cnn - mem_before_base_cnn) / 10
print(f"CPU Memory Used for Training: {mem_used_base_cnn:.2f} MB")

# if gpu_mem_before_base_cnn is not None and gpu_mem_after_base_cnn
#   gpu_mem_used_base_cnn = (gpu_mem_after_base_cnn - gpu_mem_befor

#   print(f"GPU Memory Used for Training: {gpu_mem_used_svm:.2f} MB

# else:
#   print("GPU memory info not available.")
```

```
Training Time: 370.91 sec
CPU Memory Used for Training: 732.27 MB
```

CNN - Evaluating the Model

In [31]:

```
# Evaluate the model with test dataset
loss, accuracy, precision, recall, f1_score = base_cnn_model.evalua
print(f"Test Loss for Baseline CNN Model: {loss:.4f}")
print(f"Test Accuracy for Baseline CNN Model: {accuracy * 100:.2f}%
print(f"Test Precision for Baseline CNN Model: {precision:.4f}")
print(f"Test Recall for Baseline CNN Model: {recall:.4f}")
print(f"Test F1 Score for Baseline CNN Model: {f1_score:.4f}")
```

```
71/71 ━━━━━━━━━━━━━━━━━━━━ 3s 37ms/step - accuracy: 0.5993 - f1_scor
e: 0.5907 - loss: 2.4567 - precision: 0.6607 - recall: 0.5630
Test Loss for Baseline CNN Model: 2.4270
Test Accuracy for Baseline CNN Model: 60.84%
Test Precision for Baseline CNN Model: 0.6731
Test Recall for Baseline CNN Model: 0.5739
Test F1 Score for Baseline CNN Model: 0.6116
```

In [32]:
```
# Plot the training and validation accuracy
acc = history.history['accuracy']
val_acc = history.history['val_accuracy']

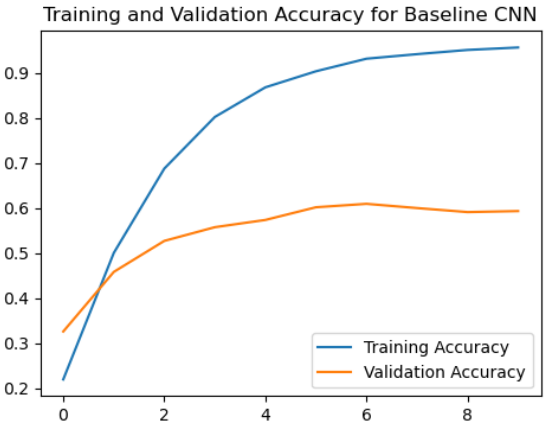
loss = history.history['loss']
val_loss = history.history['val_loss']

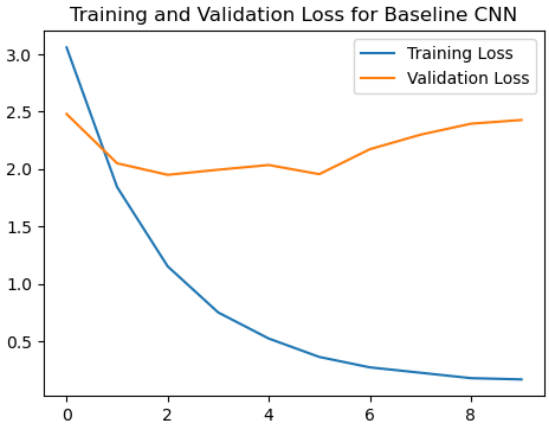
epochs_range = range(10)

plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(epochs_range, acc, label='Training Accuracy')
plt.plot(epochs_range, val_acc, label='Validation Accuracy')
plt.title('Training and Validation Accuracy for Baseline CNN')
plt.legend(loc='lower right')

plt.subplot(1, 2, 2)
plt.plot(epochs_range, loss, label='Training Loss')
plt.plot(epochs_range, val_loss, label='Validation Loss')
plt.title('Training and Validation Loss for Baseline CNN')
plt.legend(loc='upper right')

plt.show()
```





3. CNN(Residual Connection)

CNN(Residual Connection and SE-block) - Building the Model

In [33]:
```
# Input
inp = keras.layers.Input((256, 256, 3), name='input_layer')

# --- Block 1 + 2 with residual ---
# conv1 → act1 → pool1
x = layers.Conv2D(32, (3, 3), padding='same', name='conv1')(inp)
x = layers.Activation('relu', name='act1')(x)
p1 = layers.MaxPooling2D((2, 2), name='pool1')(x)

# conv2 → act2
x = layers.Conv2D(64, (3, 3), padding='same', name='conv2')(p1)
```

```
x = layers.Activation('relu', name='act2')(x)

# Projection of p1 → match channels(64)
skip1 = layers.Conv2D(64, (1, 1), padding='same', name='proj1')(p1)

# Add residual
x = layers.Add(name='residual_add1')([x, skip1])

# pool2
x = layers.MaxPooling2D((2, 2), name='pool2')(x)

# --- Block 3 + 4 with residual ---
# conv3 → act3 → pool3
x = layers.Conv2D(128, (3, 3), padding='same', name='conv3')(x)
x = layers.Activation('relu', name='act3')(x)
p3 = layers.MaxPooling2D((2, 2), name='pool3')(x)

# conv4 → act4
x = layers.Conv2D(128, (3, 3), padding='same', name='conv4')(p3)
x = layers.Activation('relu', name='act4')(x)

# Projection of p3 → match channels(128)
skip2 = layers.Conv2D(128, (1, 1), padding='same', name='proj2')(p3

# Add residual
x = layers.Add(name='residual_add2')([x, skip2])

# pool4
x = layers.MaxPooling2D((2, 2), name='pool4')(x)

# --- Classification head ---
x = layers.Flatten(name='flatten')(x)
x = layers.Dense(128, activation='relu', name='dense1')(x)
x = layers.Dropout(0.5, name='dropout1')(x)
logits = layers.Dense(30, name='output_logits')(x)
out = layers.Softmax(name='output_softmax')(logits)

# Build model
residual_model = Model(inputs=inp, outputs=out, name='residual_cnn'
residual_model.summary()
```

Model: "residual_cnn"

Layer (type)	Output Shape	Param #	Conne
input_layer (InputLayer)	(None, 256, 256, 3)	0	–
conv1 (Conv2D)	(None, 256, 256, 32)	896	input
act1 (Activation)	(None, 256, 256, 32)	0	conv1
pool1 (MaxPooling2D)	(None, 128, 128, 32)	0	act1[

conv2 (Conv2D)	(None, 128, 128, 64)	18,496	pool1
act2 (Activation)	(None, 128, 128, 64)	0	conv2
proj1 (Conv2D)	(None, 128, 128, 64)	2,112	pool1
residual_add1 (Add)	(None, 128, 128, 64)	0	act2[proj1
pool2 (MaxPooling2D)	(None, 64, 64, 64)	0	resid
conv3 (Conv2D)	(None, 64, 64, 128)	73,856	pool2
act3 (Activation)	(None, 64, 64, 128)	0	conv3
pool3 (MaxPooling2D)	(None, 32, 32, 128)	0	act3[
conv4 (Conv2D)	(None, 32, 32, 128)	147,584	pool3
act4 (Activation)	(None, 32, 32, 128)	0	conv4
proj2 (Conv2D)	(None, 32, 32, 128)	16,512	pool3
residual_add2 (Add)	(None, 32, 32, 128)	0	act4[proj2
pool4 (MaxPooling2D)	(None, 16, 16, 128)	0	resid
flatten (Flatten)	(None, 32768)	0	pool4
dense1 (Dense)	(None, 128)	4,194,432	flatt
dropout1 (Dropout)	(None, 128)	0	dense
output_logits (Dense)	(None, 30)	3,870	dropo
output_softmax (Softmax)	(None, 30)	0	outpu

Total params: 4,457,758 (17.00 MB)
Trainable params: 4,457,758 (17.00 MB)
Non-trainable params: 0 (0.00 B)

CNN(Residual Connection and SE-block) - Compile the

Model

In [34]:
```
# compile the model
residual_model.compile(
    optimizer = 'adam',
    loss = 'categorical_crossentropy',
    metrics = [
        'accuracy',
        Precision(name='precision'),
        Recall(name='recall'),
        F1Score(name='f1_score', average='macro')
    ]
)
# Define early stopping
# early_stop = EarlyStopping(
#     monitor = 'val_loss',  # Training shops when the validation e
#     patience = 3,  # Stopping the training if no improvements sho
#     restore_best_weights = True)
# # Define reduce LROn plateau
# reduce_lr = ReduceLROnPlateau(
#     monitor = 'val_loss',
#     factor = 0.5,
#     patience = 2,  # Adjusting learning rate max. twice
#     min_lr = 1e-6,
#     verbose = 1)
```

CNN(Residual Connection) - Training the Model

In [35]:
```
# Train the model
# 1) Record memory usage before training
mem_before_residaul_cnn = psutil.virtual_memory().used

# 2) Record training start time
start_time_residual_cnn = time.time()

# 3) fit the model
history_residual = residual_model.fit(
    train_cnn,
    validation_data = validation_cnn,
    epochs = 30,  # Increased from 10 to 30
    verbose = 1
)

# 4) Record training end time
end_time_residaul_cnn = time.time()

# 5) Record CPU Usage after training
mem_after_residaul_cnn = psutil.virtual_memory().used
```

```
Epoch 1/30
329/329 ━━━━━━━━━━━━━━━━━━━━ 49s 142ms/step - accuracy: 0.0657 - f1_
score: 0.0585 - loss: 3.4560 - precision: 0.1439 - recall: 0.0018 -
val_accuracy: 0.2446 - val_f1_score: 0.1975 - val_loss: 2.7325 - val
_precision: 0.7162 - val_recall: 0.0473
Epoch 2/30
329/329 ━━━━━━━━━━━━━━━━━━━━ 0s 129ms/step - accuracy: 0.2146 - f1_s
core: 0.1936 - loss: 2.8528 - precision: 0.6016 - recall: 0.0442
2025-05-12 13:00:46.903264: E tensorflow/core/grappler/optimizers/me
ta_optimizer.cc:961] PluggableGraphOptimizer failed: INVALID_ARGUMEN
T: Failed to deserialize the `graph_buf`.
329/329 ━━━━━━━━━━━━━━━━━━━━ 46s 139ms/step - accuracy: 0.2146 - f1_
score: 0.1937 - loss: 2.8525 - precision: 0.6017 - recall: 0.0443 -
val_accuracy: 0.3259 - val_f1_score: 0.3080 - val_loss: 2.3967 - val
_precision: 0.7460 - val_recall: 0.1049
Epoch 3/30
329/329 ━━━━━━━━━━━━━━━━━━━━ 45s 137ms/step - accuracy: 0.3021 - f1_
score: 0.2889 - loss: 2.4832 - precision: 0.6948 - recall: 0.1165 -
val_accuracy: 0.3759 - val_f1_score: 0.3632 - val_loss: 2.1767 - val
_precision: 0.7543 - val_recall: 0.1973
Epoch 4/30
329/329 ━━━━━━━━━━━━━━━━━━━━ 0s 128ms/step - accuracy: 0.3935 - f1_s
core: 0.3817 - loss: 2.1833 - precision: 0.7233 - recall: 0.1905
2025-05-12 13:02:17.156451: E tensorflow/core/grappler/optimizers/me
ta_optimizer.cc:961] PluggableGraphOptimizer failed: INVALID_ARGUMEN
T: Failed to deserialize the `graph_buf`.
329/329 ━━━━━━━━━━━━━━━━━━━━ 45s 137ms/step - accuracy: 0.3935 - f1_
score: 0.3817 - loss: 2.1831 - precision: 0.7234 - recall: 0.1906 -
val_accuracy: 0.4232 - val_f1_score: 0.4132 - val_loss: 2.0463 - val
_precision: 0.7645 - val_recall: 0.2174
Epoch 5/30
329/329 ━━━━━━━━━━━━━━━━━━━━ 46s 138ms/step - accuracy: 0.4478 - f1_
score: 0.4398 - loss: 1.9463 - precision: 0.7479 - recall: 0.2528 -
val_accuracy: 0.4777 - val_f1_score: 0.4719 - val_loss: 1.8452 - val
_precision: 0.7656 - val_recall: 0.3063
Epoch 6/30
329/329 ━━━━━━━━━━━━━━━━━━━━ 0s 131ms/step - accuracy: 0.5138 - f1_s
core: 0.5076 - loss: 1.6694 - precision: 0.7754 - recall: 0.3481
2025-05-12 13:03:49.079229: E tensorflow/core/grappler/optimizers/me
ta_optimizer.cc:961] PluggableGraphOptimizer failed: INVALID_ARGUMEN
T: Failed to deserialize the `graph_buf`.
329/329 ━━━━━━━━━━━━━━━━━━━━ 46s 140ms/step - accuracy: 0.5138 - f1_
score: 0.5077 - loss: 1.6693 - precision: 0.7754 - recall: 0.3482 -
val_accuracy: 0.5254 - val_f1_score: 0.5150 - val_loss: 1.7210 - val
_precision: 0.8082 - val_recall: 0.3857
Epoch 7/30
329/329 ━━━━━━━━━━━━━━━━━━━━ 46s 138ms/step - accuracy: 0.5589 - f1_
score: 0.5523 - loss: 1.5180 - precision: 0.7900 - recall: 0.3963 -
val_accuracy: 0.5554 - val_f1_score: 0.5498 - val_loss: 1.6285 - val
_precision: 0.7762 - val_recall: 0.4040
Epoch 8/30
329/329 ━━━━━━━━━━━━━━━━━━━━ 0s 129ms/step - accuracy: 0.5947 - f1_s
core: 0.5902 - loss: 1.3682 - precision: 0.8087 - recall: 0.4503
```

```
2025-05-12 13:05:20.238735: E tensorflow/core/grappler/optimizers/meta_optimizer.cc:961] PluggableGraphOptimizer failed: INVALID_ARGUMENT: Failed to deserialize the `graph_buf`.
329/329 ━━━━━━━━━━━━━━━━━━━━ 46s 138ms/step - accuracy: 0.5948 - f1_score: 0.5902 - loss: 1.3680 - precision: 0.8087 - recall: 0.4504 - val_accuracy: 0.5688 - val_f1_score: 0.5609 - val_loss: 1.5954 - val_precision: 0.8048 - val_recall: 0.4308
Epoch 9/30
329/329 ━━━━━━━━━━━━━━━━━━━━ 45s 138ms/step - accuracy: 0.6376 - f1_score: 0.6332 - loss: 1.2256 - precision: 0.8110 - recall: 0.5060 - val_accuracy: 0.5955 - val_f1_score: 0.5918 - val_loss: 1.5746 - val_precision: 0.7622 - val_recall: 0.4951
Epoch 10/30
329/329 ━━━━━━━━━━━━━━━━━━━━ 0s 132ms/step - accuracy: 0.6815 - f1_score: 0.6769 - loss: 1.0671 - precision: 0.8356 - recall: 0.5714
2025-05-12 13:06:52.190881: E tensorflow/core/grappler/optimizers/meta_optimizer.cc:961] PluggableGraphOptimizer failed: INVALID_ARGUMENT: Failed to deserialize the `graph_buf`.
329/329 ━━━━━━━━━━━━━━━━━━━━ 47s 141ms/step - accuracy: 0.6815 - f1_score: 0.6769 - loss: 1.0670 - precision: 0.8356 - recall: 0.5715 - val_accuracy: 0.6031 - val_f1_score: 0.5961 - val_loss: 1.5699 - val_precision: 0.7606 - val_recall: 0.5263
Epoch 11/30
329/329 ━━━━━━━━━━━━━━━━━━━━ 45s 138ms/step - accuracy: 0.7009 - f1_score: 0.6980 - loss: 0.9714 - precision: 0.8294 - recall: 0.6026 - val_accuracy: 0.6054 - val_f1_score: 0.6036 - val_loss: 1.5580 - val_precision: 0.7772 - val_recall: 0.5107
Epoch 12/30
329/329 ━━━━━━━━━━━━━━━━━━━━ 0s 127ms/step - accuracy: 0.7280 - f1_score: 0.7241 - loss: 0.8581 - precision: 0.8492 - recall: 0.6446
2025-05-12 13:08:22.443149: E tensorflow/core/grappler/optimizers/meta_optimizer.cc:961] PluggableGraphOptimizer failed: INVALID_ARGUMENT: Failed to deserialize the `graph_buf`.
329/329 ━━━━━━━━━━━━━━━━━━━━ 45s 136ms/step - accuracy: 0.7280 - f1_score: 0.7241 - loss: 0.8581 - precision: 0.8492 - recall: 0.6446 - val_accuracy: 0.6179 - val_f1_score: 0.6169 - val_loss: 1.6228 - val_precision: 0.7495 - val_recall: 0.5504
Epoch 13/30
329/329 ━━━━━━━━━━━━━━━━━━━━ 46s 139ms/step - accuracy: 0.7488 - f1_score: 0.7455 - loss: 0.8150 - precision: 0.8506 - recall: 0.6663 - val_accuracy: 0.6308 - val_f1_score: 0.6255 - val_loss: 1.6080 - val_precision: 0.7595 - val_recall: 0.5683
Epoch 14/30
329/329 ━━━━━━━━━━━━━━━━━━━━ 0s 129ms/step - accuracy: 0.7817 - f1_score: 0.7784 - loss: 0.7107 - precision: 0.8722 - recall: 0.7167
2025-05-12 13:09:53.712973: E tensorflow/core/grappler/optimizers/meta_optimizer.cc:961] PluggableGraphOptimizer failed: INVALID_ARGUMENT: Failed to deserialize the `graph_buf`.
```

```
329/329 ━━━━━━━━━━━━━━━━━━━━ 46s 138ms/step - accuracy: 0.7817 - f1_
score: 0.7784 - loss: 0.7106 - precision: 0.8722 - recall: 0.7167 -
val_accuracy: 0.6415 - val_f1_score: 0.6351 - val_loss: 1.5396 - val
_precision: 0.7677 - val_recall: 0.5902
Epoch 15/30
329/329 ━━━━━━━━━━━━━━━━━━━━ 45s 136ms/step - accuracy: 0.7802 - f1_
score: 0.7770 - loss: 0.6975 - precision: 0.8681 - recall: 0.7100 -
val_accuracy: 0.6415 - val_f1_score: 0.6383 - val_loss: 1.6197 - val
_precision: 0.7521 - val_recall: 0.5946
Epoch 16/30
329/329 ━━━━━━━━━━━━━━━━━━━━ 0s 126ms/step - accuracy: 0.7882 - f1_s
core: 0.7849 - loss: 0.6577 - precision: 0.8672 - recall: 0.7291
2025-05-12 13:11:23.238026: E tensorflow/core/grappler/optimizers/me
ta_optimizer.cc:961] PluggableGraphOptimizer failed: INVALID_ARGUMEN
T: Failed to deserialize the `graph_buf`.
329/329 ━━━━━━━━━━━━━━━━━━━━ 45s 136ms/step - accuracy: 0.7883 - f1_
score: 0.7849 - loss: 0.6576 - precision: 0.8672 - recall: 0.7292 -
val_accuracy: 0.6290 - val_f1_score: 0.6258 - val_loss: 1.7608 - val
_precision: 0.7335 - val_recall: 0.5911
Epoch 17/30
329/329 ━━━━━━━━━━━━━━━━━━━━ 46s 138ms/step - accuracy: 0.7922 - f1_
score: 0.7892 - loss: 0.6452 - precision: 0.8693 - recall: 0.7343 -
val_accuracy: 0.6335 - val_f1_score: 0.6301 - val_loss: 1.8781 - val
_precision: 0.7184 - val_recall: 0.6071
Epoch 18/30
329/329 ━━━━━━━━━━━━━━━━━━━━ 0s 128ms/step - accuracy: 0.8138 - f1_s
core: 0.8103 - loss: 0.5675 - precision: 0.8770 - recall: 0.7658
2025-05-12 13:12:54.165981: E tensorflow/core/grappler/optimizers/me
ta_optimizer.cc:961] PluggableGraphOptimizer failed: INVALID_ARGUMEN
T: Failed to deserialize the `graph_buf`.
329/329 ━━━━━━━━━━━━━━━━━━━━ 45s 137ms/step - accuracy: 0.8138 - f1_
score: 0.8104 - loss: 0.5675 - precision: 0.8770 - recall: 0.7658 -
val_accuracy: 0.6482 - val_f1_score: 0.6425 - val_loss: 1.7505 - val
_precision: 0.7409 - val_recall: 0.6152
Epoch 19/30
329/329 ━━━━━━━━━━━━━━━━━━━━ 45s 138ms/step - accuracy: 0.8148 - f1_
score: 0.8101 - loss: 0.5779 - precision: 0.8747 - recall: 0.7674 -
val_accuracy: 0.6473 - val_f1_score: 0.6460 - val_loss: 1.8090 - val
_precision: 0.7379 - val_recall: 0.6045
Epoch 20/30
329/329 ━━━━━━━━━━━━━━━━━━━━ 0s 130ms/step - accuracy: 0.8259 - f1_s
core: 0.8237 - loss: 0.5118 - precision: 0.8818 - recall: 0.7890
2025-05-12 13:14:25.545623: E tensorflow/core/grappler/optimizers/me
ta_optimizer.cc:961] PluggableGraphOptimizer failed: INVALID_ARGUMEN
T: Failed to deserialize the `graph_buf`.
```

```
329/329 ━━━━━━━━━━━━━━━━━━━━ 46s 140ms/step - accuracy: 0.8259 - f1_
score: 0.8237 - loss: 0.5118 - precision: 0.8818 - recall: 0.7890 -
val_accuracy: 0.6504 - val_f1_score: 0.6478 - val_loss: 1.7970 - val
_precision: 0.7370 - val_recall: 0.6179
Epoch 21/30
329/329 ━━━━━━━━━━━━━━━━━━━━ 45s 135ms/step - accuracy: 0.8423 - f1_
score: 0.8382 - loss: 0.4860 - precision: 0.8911 - recall: 0.8009 -
val_accuracy: 0.6420 - val_f1_score: 0.6432 - val_loss: 1.8001 - val
_precision: 0.7363 - val_recall: 0.6071
Epoch 22/30
329/329 ━━━━━━━━━━━━━━━━━━━━ 0s 126ms/step - accuracy: 0.8438 - f1_s
core: 0.8415 - loss: 0.4897 - precision: 0.8902 - recall: 0.8072
2025-05-12 13:15:54.585108: E tensorflow/core/grappler/optimizers/me
ta_optimizer.cc:961] PluggableGraphOptimizer failed: INVALID_ARGUMEN
T: Failed to deserialize the `graph_buf`.
329/329 ━━━━━━━━━━━━━━━━━━━━ 44s 135ms/step - accuracy: 0.8438 - f1_
score: 0.8415 - loss: 0.4897 - precision: 0.8902 - recall: 0.8072 -
val_accuracy: 0.6504 - val_f1_score: 0.6491 - val_loss: 1.7939 - val
_precision: 0.7353 - val_recall: 0.6201
Epoch 23/30
329/329 ━━━━━━━━━━━━━━━━━━━━ 44s 134ms/step - accuracy: 0.8501 - f1_
score: 0.8474 - loss: 0.4503 - precision: 0.8962 - recall: 0.8177 -
val_accuracy: 0.6451 - val_f1_score: 0.6435 - val_loss: 1.9625 - val
_precision: 0.7213 - val_recall: 0.6170
Epoch 24/30
329/329 ━━━━━━━━━━━━━━━━━━━━ 0s 126ms/step - accuracy: 0.8441 - f1_s
core: 0.8411 - loss: 0.4967 - precision: 0.8852 - recall: 0.8072
2025-05-12 13:17:23.235124: E tensorflow/core/grappler/optimizers/me
ta_optimizer.cc:961] PluggableGraphOptimizer failed: INVALID_ARGUMEN
T: Failed to deserialize the `graph_buf`.
329/329 ━━━━━━━━━━━━━━━━━━━━ 44s 135ms/step - accuracy: 0.8441 - f1_
score: 0.8412 - loss: 0.4967 - precision: 0.8852 - recall: 0.8072 -
val_accuracy: 0.6469 - val_f1_score: 0.6461 - val_loss: 2.0029 - val
_precision: 0.7209 - val_recall: 0.6250
Epoch 25/30
329/329 ━━━━━━━━━━━━━━━━━━━━ 44s 134ms/step - accuracy: 0.8566 - f1_
score: 0.8535 - loss: 0.4502 - precision: 0.8981 - recall: 0.8217 -
val_accuracy: 0.6478 - val_f1_score: 0.6469 - val_loss: 1.8986 - val
_precision: 0.7298 - val_recall: 0.6138
Epoch 26/30
329/329 ━━━━━━━━━━━━━━━━━━━━ 0s 125ms/step - accuracy: 0.8528 - f1_s
core: 0.8496 - loss: 0.4492 - precision: 0.8961 - recall: 0.8208
2025-05-12 13:18:51.741823: E tensorflow/core/grappler/optimizers/me
ta_optimizer.cc:961] PluggableGraphOptimizer failed: INVALID_ARGUMEN
T: Failed to deserialize the `graph_buf`.
```

```
329/329 ━━━━━━━━━━━━━━━━━━━━ 44s 134ms/step - accuracy: 0.8528 - f1_
score: 0.8496 - loss: 0.4492 - precision: 0.8961 - recall: 0.8209 -
val_accuracy: 0.6429 - val_f1_score: 0.6400 - val_loss: 1.9672 - val
_precision: 0.7149 - val_recall: 0.6179
Epoch 27/30
329/329 ━━━━━━━━━━━━━━━━━━━━ 45s 137ms/step - accuracy: 0.8548 - f1_
score: 0.8519 - loss: 0.4450 - precision: 0.8928 - recall: 0.8238 -
val_accuracy: 0.6496 - val_f1_score: 0.6455 - val_loss: 1.9155 - val
_precision: 0.7242 - val_recall: 0.6237
Epoch 28/30
329/329 ━━━━━━━━━━━━━━━━━━━━ 0s 126ms/step - accuracy: 0.8693 - f1_s
core: 0.8663 - loss: 0.3943 - precision: 0.9053 - recall: 0.8422
2025-05-12 13:20:21.410413: E tensorflow/core/grappler/optimizers/me
ta_optimizer.cc:961] PluggableGraphOptimizer failed: INVALID_ARGUMEN
T: Failed to deserialize the `graph_buf`.
329/329 ━━━━━━━━━━━━━━━━━━━━ 45s 135ms/step - accuracy: 0.8693 - f1_
score: 0.8663 - loss: 0.3943 - precision: 0.9053 - recall: 0.8421 -
val_accuracy: 0.6562 - val_f1_score: 0.6534 - val_loss: 1.8337 - val
_precision: 0.7419 - val_recall: 0.6326
Epoch 29/30
329/329 ━━━━━━━━━━━━━━━━━━━━ 44s 135ms/step - accuracy: 0.8664 - f1_
score: 0.8641 - loss: 0.3863 - precision: 0.9037 - recall: 0.8430 -
val_accuracy: 0.6536 - val_f1_score: 0.6517 - val_loss: 2.0198 - val
_precision: 0.7199 - val_recall: 0.6299
Epoch 30/30
329/329 ━━━━━━━━━━━━━━━━━━━━ 0s 125ms/step - accuracy: 0.8712 - f1_s
core: 0.8695 - loss: 0.3894 - precision: 0.9018 - recall: 0.8443
2025-05-12 13:21:50.167907: E tensorflow/core/grappler/optimizers/me
ta_optimizer.cc:961] PluggableGraphOptimizer failed: INVALID_ARGUMEN
T: Failed to deserialize the `graph_buf`.
329/329 ━━━━━━━━━━━━━━━━━━━━ 44s 134ms/step - accuracy: 0.8712 - f1_
score: 0.8695 - loss: 0.3894 - precision: 0.9018 - recall: 0.8443 -
val_accuracy: 0.6429 - val_f1_score: 0.6387 - val_loss: 1.9612 - val
_precision: 0.7179 - val_recall: 0.6237
```

In [36]:
```python
# 6) Training time used
train_duration_residaul_cnn = end_time_residaul_cnn - start_time_re
print(f"Training Time: {train_duration_residaul_cnn:.2f} sec")

# 7) CPU usage
mem_used_residaul_cnn = (mem_after_residaul_cnn - mem_before_resida
print(f"Memory Used for Training: {mem_used_residaul_cnn:.2f} MB(CP
```

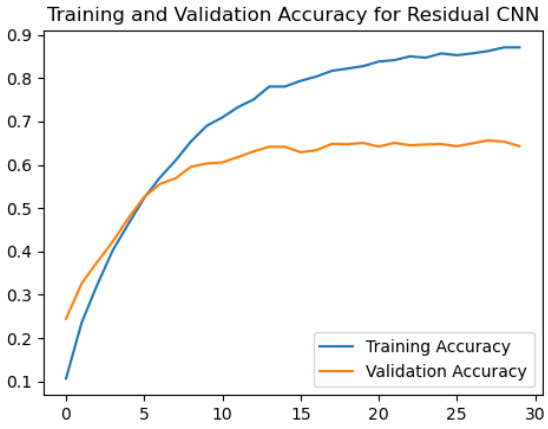
```
Training Time: 1357.95 sec
Memory Used for Training: -510.16 MB(CPU)
```

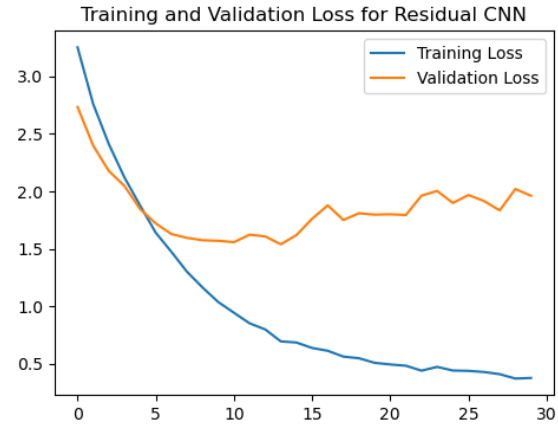
CNN(Residual Connection) - Evaluating the Model

In [37]:
```python
# Evaluate the model with test dataset
loss, accuracy, precision, recall, f1_score = residual_model.evalua
print(f"Test Loss for Residual Model: {loss:.4f}")
print(f"Test Accuracy for Residual CNN Model: {accuracy * 100:.2f}%
print(f"Test Precision for Residual CNN Model: {precision:.4f}")
print(f"Test Recall for Residual CNN Model: {recall:.4f}")
print(f"Test F1 Score for Residual CNN Model: {f1_score:.4f}")
```

```
71/71 ━━━━━━━━━━━━━━━━━━━━ 4s 47ms/step - accuracy: 0.6467 - f1_scor
e: 0.6358 - loss: 1.9760 - precision: 0.7187 - recall: 0.6278
Test Loss for Residual Model: 1.8949
Test Accuracy for Residual CNN Model: 66.11%
Test Precision for Residual CNN Model: 0.7342
Test Recall for Residual CNN Model: 0.6416
Test F1 Score for Residual CNN Model: 0.6606
```

In [38]:
```python
# plot the training and validation accuracy
acc = history_residual.history['accuracy']
val_acc = history_residual.history['val_accuracy']
loss = history_residual.history['loss']
val_loss = history_residual.history['val_loss']
epochs_range = range(len(acc))
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(epochs_range, acc, label='Training Accuracy')
plt.plot(epochs_range, val_acc, label='Validation Accuracy')
plt.title('Training and Validation Accuracy for Residual CNN')
plt.legend(loc='lower right')
plt.subplot(1, 2, 2)
plt.plot(epochs_range, loss, label='Training Loss')
plt.plot(epochs_range, val_loss, label='Validation Loss')
plt.title('Training and Validation Loss for Residual CNN')
plt.legend(loc='upper right')
plt.show()
```





4. EfficientNet Model with Fine Tuning

EfficientNet Model with Fine Tuning - Building the Model

In [39]:
```python
# 1-1) Build base EfficientNet model
base_model = tf.keras.applications.EfficientNetB0(
    input_shape=(256, 256, 3),
    include_top=False,
    weights='imagenet'
)

# 1-2) Keep the pre-trained model by default
base_model.trainable = False # Freeze the base model
```

```
# 2) Fine tune the model by unfreezing the last 20 layers of the ba
for layer in base_model.layers[-20:]:
    layer.trainable = True

# 3) Customise the base model
# 1st - Data augmentation
efficientnet_model_ft = Sequential()
# efficientnet_model_ft.add(RandomFlip('horizontal', input_shape=(2
# efficientnet_model_ft.add(RandomRotation(0.1))
# efficientnet_model_ft.add(RandomZoom(0.1))
# efficientnet_model_ft.add(RandomTranslation(0.1, 0.1))

# 2nd - Base model
efficientnet_model_ft.add(base_model)

# 3rd - Flatten + Dropout + Dense
efficientnet_model_ft.add(GlobalAveragePooling2D())
efficientnet_model_ft.add(Dropout(0.5))
efficientnet_model_ft.add(Dense(30, activation='softmax'))

# 4) Model summary
efficientnet_model_ft.summary()
```

Model: "sequential"

Layer (type)	Output Shape	
efficientnetb0 (Functional)	(None, 8, 8, 1280)	
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1280)	
dropout (Dropout)	(None, 1280)	
dense (Dense)	(None, 30)	

Total params: 4,088,001 (15.59 MB)

Trainable params: 1,389,390 (5.30 MB)

Non-trainable params: 2,698,611 (10.29 MB)

EfficientNet Model with Fine Tuning - Compiling the Model

In [40]:
```
# Compile the model
efficientnet_model_ft.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=[
        'accuracy',
        Precision(name='precision'),
        Recall(name='recall'),
        F1Score(name='f1_score', average='macro')
```

```
    ]
)
```

EfficientNet Model with Fine Tuning - Training the Model

In [41]:
```
# Define early stopping
earlystop_efficientnet_ft = EarlyStopping(patience=5, restore_best_

# Define reduce LROn plateau
reduce_lr_efficientnet_ft = ReduceLROnPlateau(patience=3, factor=0.

# Combine for callbacks
callbacks_efficientnet_ft = [earlystop_efficientnet_ft, reduce_lr_e

# Train the model
# 1) Record memory usage before training
mem_before_efficientnet_ft = psutil.virtual_memory().used

# 2) Record training start time
start_time_efficientnet_ft = time.time()

# 3) Fit the data into efficient model
history_efficientnet_ft = efficientnet_model_ft.fit(
    train,
    validation_data = validation,
    epochs = 20,
    callbacks = callbacks_efficientnet_ft,
    verbose = 1
)

# 4) Record training end time
end_time_efficientnet_ft = time.time()

# 5) Record CPU Usage after training
mem_after_efficientnet_ft = psutil.virtual_memory().used
```

```
Epoch 1/20
329/329 ━━━━━━━━━━━━━━━━━━━━ 0s 153ms/step - accuracy: 0.6271 - f1_s
core: 0.6196 - loss: 1.3138 - precision: 0.8402 - recall: 0.4863
2025-05-12 13:22:59.509661: E tensorflow/core/grappler/optimizers/me
ta_optimizer.cc:961] PluggableGraphOptimizer failed: INVALID_ARGUMEN
T: Failed to deserialize the `graph_buf`.
```

```
329/329 ━━━━━━━━━━━━━━━━━━━━ 72s 190ms/step - accuracy: 0.6274 - f1_
score: 0.6199 - loss: 1.3124 - precision: 0.8402 - recall: 0.4868 -
val_accuracy: 0.8348 - val_f1_score: 0.8350 - val_loss: 0.5278 - val
_precision: 0.8657 - val_recall: 0.8000 - learning_rate: 0.0010
Epoch 2/20
Epoch 2/20
329/329 ━━━━━━━━━━━━━━━━━━━━ 58s 174ms/step - accuracy: 0.8578 - f1_
score: 0.8543 - loss: 0.4506 - precision: 0.9005 - recall: 0.8106 -
val_accuracy: 0.8540 - val_f1_score: 0.8510 - val_loss: 0.4311 - val
_precision: 0.8843 - val_recall: 0.8295 - learning_rate: 0.0010
Epoch 3/20
329/329 ━━━━━━━━━━━━━━━━━━━━ 59s 179ms/step - accuracy: 0.8914 - f1_
score: 0.8878 - loss: 0.3063 - precision: 0.9133 - recall: 0.8665 -
val_accuracy: 0.8598 - val_f1_score: 0.8564 - val_loss: 0.4258 - val
_precision: 0.8785 - val_recall: 0.8455 - learning_rate: 0.0010
Epoch 4/20
329/329 ━━━━━━━━━━━━━━━━━━━━ 58s 175ms/step - accuracy: 0.9127 - f1_
score: 0.9097 - loss: 0.2362 - precision: 0.9286 - recall: 0.8956 -
val_accuracy: 0.8567 - val_f1_score: 0.8524 - val_loss: 0.4965 - val
_precision: 0.8723 - val_recall: 0.8446 - learning_rate: 0.0010
Epoch 5/20
329/329 ━━━━━━━━━━━━━━━━━━━━ 58s 175ms/step - accuracy: 0.9286 - f1_
score: 0.9253 - loss: 0.1855 - precision: 0.9373 - recall: 0.9212 -
val_accuracy: 0.8589 - val_f1_score: 0.8545 - val_loss: 0.5087 - val
_precision: 0.8724 - val_recall: 0.8487 - learning_rate: 0.0010
Epoch 6/20
329/329 ━━━━━━━━━━━━━━━━━━━━ 57s 174ms/step - accuracy: 0.9417 - f1_
score: 0.9393 - loss: 0.1612 - precision: 0.9498 - recall: 0.9334 -
val_accuracy: 0.8612 - val_f1_score: 0.8573 - val_loss: 0.4919 - val
_precision: 0.8768 - val_recall: 0.8513 - learning_rate: 0.0010
Epoch 7/20
329/329 ━━━━━━━━━━━━━━━━━━━━ 58s 175ms/step - accuracy: 0.9505 - f1_
score: 0.9479 - loss: 0.1312 - precision: 0.9549 - recall: 0.9450 -
val_accuracy: 0.8754 - val_f1_score: 0.8705 - val_loss: 0.4358 - val
_precision: 0.8900 - val_recall: 0.8701 - learning_rate: 5.0000e-04
Epoch 8/20
329/329 ━━━━━━━━━━━━━━━━━━━━ 58s 176ms/step - accuracy: 0.9592 - f1_
score: 0.9557 - loss: 0.1025 - precision: 0.9620 - recall: 0.9549 -
val_accuracy: 0.8808 - val_f1_score: 0.8788 - val_loss: 0.4504 - val
_precision: 0.8968 - val_recall: 0.8728 - learning_rate: 5.0000e-04
```

In [42]:
```python
# 6) Training Time Used
train_duration_efficientnet_ft = end_time_efficientnet_ft - start_t
print(f"Training Time: {train_duration_efficientnet_ft:.2f} sec")

# 7) Record CPU Usage after training
mem_used_efficientnet_ft = (mem_after_efficientnet_ft  - mem_before
print(f"Memory Used for Training: {mem_used_efficientnet_ft:.2f} MB
```

```
Training Time: 476.92 sec
Memory Used for Training: 1072.05 MB (CPU)
```

EfficientNet Model with Fine Tuning - Evaluating the Model

In [43]:
```
# Evaluate the model with test dataset
loss, accuracy, precision, recall, f1_score = efficientnet_model_ft
print(f"Test Loss for EfficientNet Model: {loss:.4f}")
print(f"Test Accuracy for EfficientNet CNN Model: {accuracy * 100:.
print(f"Test Precision for EfficientNet CNN Model: {precision:.4f}"
print(f"Test Recall for EfficientNet CNN Model: {recall:.4f}")
print(f"Test F1 Score for EfficientNet CNN Model: {f1_score:.4f}")
```

```
71/71 ━━━━━━━━━━━━━━━━━━━━ 11s 142ms/step - accuracy: 0.8770 - f1_sc
ore: 0.8651 - loss: 0.3885 - precision: 0.8925 - recall: 0.8673
Test Loss for EfficientNet Model: 0.4058
Test Accuracy for EfficientNet CNN Model: 87.39%
Test Precision for EfficientNet CNN Model: 0.8918
Test Recall for EfficientNet CNN Model: 0.8642
Test F1 Score for EfficientNet CNN Model: 0.8734
```

In [44]:
```
# Plot the training and validation accuracy
acc = history_efficientnet_ft.history['accuracy']
val_acc = history_efficientnet_ft.history['val_accuracy']

loss = history_efficientnet_ft.history['loss']
val_loss = history_efficientnet_ft.history['val_loss']

epochs_range = range(len(acc))

plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(epochs_range, acc, label='Training Accuracy')
plt.plot(epochs_range, val_acc, label='Validation Accuracy')
plt.title('Training and Validation Accuracy for EfficientNet with F
plt.legend(loc='lower right')

plt.subplot(1, 2, 2)
plt.plot(epochs_range, loss, label='Training Loss')
plt.plot(epochs_range, val_loss, label='Validation Loss')
plt.title('Training and Validation Loss for EfficientNet with Fine-
plt.legend(loc='upper right')

plt.show()
```

